

Certificates and TLS, Self-Guided Version

Cyber Bootcamp 2026

August 28.2026

By: Jason Man

Part 1, Day 4: Build Your Own Certificate Authority and Set Up TLS

Step 1: Create a working folder

```
C:\Users\carin>mkdir week4-tls-lab
A subdirectory or file week4-tls-lab already exists.

C:\Users\carin>cd week4-tls-lab

C:\Users\carin\week4-tls-lab>|
```

Step 2: Build your root Certificate Authority

```
manom@HomePC2026 MINGW64 ~/week4-tls-lab
$ openssl req -x509 -new -key ca.key -sha256 -days 3650 -out ca.crt -addext "basicConstraints=critical,CA:TRUE" -addext "keyUsage=critical,key
rtSign,cRLSign"
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:Pennsylvania
Locality Name (eg, city) []:Pittsburgh
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Meridian Trust Banking
Organizational Unit Name (eg, section) []:IT
Common Name (e.g. server FQDN or YOUR name) []:MeridianTrust-Lab-CA
Email Address []:user@example.com

manom@HomePC2026 MINGW64 ~/week4-tls-lab
$ openssl x509 -in ca.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            58:79:0d:25:30:39:2d:c5:66:09:24:8c:ae:11:b6:a4:13:52:6c:79
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=IT, CN=MeridianTrust-Lab-CA, emailAddress=user@example.com
        Validity
            Not Before: Aug 28 16:13:58 2026 GMT
            Not After : Aug 25 16:13:58 2036 GMT
        Subject: C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=IT, CN=MeridianTrust-Lab-CA, emailAddress=user@example.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
                00:a0:2f:db:3b:f0:c9:be:48:e4:55:5d:6b:5c:8b:
                04:5f:53:3d:5a:b1:57:6b:44:99:a2:ad:84:19:04:
                4e:56:97:f6:28:e7:b7:db:d3:3b:1f:67:ec:db:ad:
                9e:b6:0c:9a:29:26:89:27:b7:81:27:a0:04:da:82:
                41:d5:b6:92:33:e9:2f:ea:42:8e:56:98:88:0e:98:
                41:60:a8:94:32:78:04:3a:3b:a0:75:1d:b2:06:48:
                d6:de:cc:6e:56:40:e7:6f:24:f8:3c:61:04:a8:36:
                f1:3b:17:ec:54:db:39:ee:f3:1d:1c:63:8d:ca:f7:
                c9:c2:7e:bd:84:50:6f:cb:5c:24:b3:e4:dc:c5:86:
                40:c1:79:36:82:d5:96:cd:8e:9a:e9:7d:9e:89:36:
                eb:37:45:da:c4:bc:d8:0c:05:cd:e7:26:0b:bd:11:
                de:74:3c:7f:12:21:9d:43:28:bc:f1:c2:ec:8c:50:
            Exponent: 65537 (0x10001)
    X509v3 Basic Constraints: critical
        CA:TRUE
```

Step 3: Create a Certificate Signing request (CSR) for your test Server

```
manom0HomePC2026 MINGW64 ~/week4-tls-lab
$ openssl req -new -newkey rsa:2048 -nodes -keyout server.key -out server.csr
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:Pennsylvania
Locality Name (eg, city) []:Pittsburgh
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Meridian Trust Banking
Organizational Unit Name (eg, section) []:SOC
Common Name (e.g. server FQDN or YOUR name) []:localhost
Email Address []:user2@example.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:password
An optional company name []:
```

Step 4: Sign the CSR with your CA to issue the certificate

```

marion@HomePC2026 MINGW64 ~/week4-tls-lab
$ openssl x509 -in server.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            12:6a:53:4c:af:90:cc:5d:72:48:79:60:e3:9b:81:34:10:c5:75:44
        Signature Algorithm: sha256withRSAEncryption
        Issuer: C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=IT, CN=MeridianTrust-Lab-CA, emailAddress=user@example.com
        Validity
            Not Before: Aug 28 18:02:58 2026 GMT
            Not After : Aug 28 18:02:58 2027 GMT
        Subject: C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=SOC, CN=localhost, emailAddress=user2@example.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:9c:7a:fd:15:6f:d7:c0:6e:78:60:d1:37:b0:06:
                35:d0:de:eb:fc:f8:90:63:95:8d:a4:fa:72:00:e6:
                69:da:52:df:c1:cb:7d:ac:cd:06:7d:7c:65:3a:cd:
                d1:a7:0b:36:0e:90:47:0b:81:8f:f4:22:81:b3:f4:
                59:20:1c:fc:fe:1a:18:27:cc:7e:7a:c5:04:38:5e:
                13:76:a2:59:df:5e:bf:85:f8:d5:b0:8b:db:a8:1b:
                93:1a:79:7f:ec:ae:9c:96:2b:ac:b2:24:0d:00:a5:
                ad:4e:03:3c:a5:39:b6:70:04:d1:ad:14:30:ff:74:
                cf:c2:e2:39:a9:6e:4f:64:75:51:d6:07:eb:4e:54:
                81:52:3e:24:f9:1f:76:db:d2:f6:f7:36:c5:76:5c:
                a2:af:56:e3:ac:3b:e4:50:c0:a3:ad:93:6c:b9:06:
                7b:a8:85:b8:4c:23:af:f3:81:44:07:e4:08:d7:b0:
                fb:c5:a8:e1:76:f5:dc:1b:be:aa:cf:f6:93:be:79:
                dc:bd:b7:b1:17:f1:b0:06:75:b2:0f:d9:58:85:6c:
                f4:32:64:05:2c:c7:6b:52:00:0f:da:dc:34:8e:e3:
                e7:07:ba:40:c8:29:28:ca:7e:fb:a9:8d:b9:f0:41:
                a1:f7:17:dc:1c:c1:c2:04:6b:9c:41:fb:0d:f7:0b:
                6f:ab
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                A3:02:33:AF:30:36:B5:3C:97:28:D8:9E:43:6B:E0:5C:88:37:C5:7F
            X509v3 Authority Key Identifier:
                3C:92:D7:36:66:2C:E0:6E:A9:13:90:8A:88:75:D8:95:45:3F:2F:00
        Signature Algorithm: sha256withRSAEncryption
        Signature Value:

```

Step 5: Start a test web server using your certificate

```

carin@Borg MINGW64 ~/week4-tls-lab
$ python --version
Python 3.12.10

```

```

carin@Borg MINGW64 ~/week4-tls-lab
$ cat > https_server.py << 'EOF'
import http.server, ssl
server_address = ('0.0.0.0', 4433)
httpd = http.server.HTTPServer(server_address,
http.server.SimpleHTTPRequestHandler)
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile="server.crt", keyfile="server.key")
httpd.socket = context.wrap_socket(httpd.socket, server_side=True)
print("Serving HTTPS on port 4433...")
httpd.serve_forever()
EOF

```

```

$ python https_server.py
Serving HTTPS on port 4433...

```

Step 6: Connect to your server and inspect the certificate chain

```
$ openssl s_client -connect localhost:4433 -showcerts
Connecting to 127.0.0.1
CONNECTED(000001EC)
Can't use SSL_get_servername
depth=0 C=US, ST=Pennsylvania, L=Pittsburgh, O= Meridian Trust Bank, OU=MeridianTrustBank-PA, CN=localhost, emailAddress=carinlovespandaz@gmail.com
verify error:num=20:unable to get local issuer certificate
verify return:1
depth=0 C=US, ST=Pennsylvania, L=Pittsburgh, O= Meridian Trust Bank, OU=MeridianTrustBank-PA, CN=localhost, emailAddress=carinlovespandaz@gmail.com
verify error:num=21:unable to verify the first certificate
verify return:1
depth=0 C=US, ST=Pennsylvania, L=Pittsburgh, O= Meridian Trust Bank, OU=MeridianTrustBank-PA, CN=localhost, emailAddress=carinlovespandaz@gmail.com
verify return:1
---
Certificate chain
 0 s:C=US, ST=Pennsylvania, L=Pittsburgh, O= Meridian Trust Bank, OU=MeridianTrustBank-PA, CN=localhost, emailAddress=carinlovespandaz@gmail.com
  i:C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Bank, OU=IT services, CN=MeridianTrust-Lab-PA, emailAddress=carinlovespandaz@gmail.com
  a:PKEY: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
  v:NotBefore: Aug 28 18:05:52 2026 GMT; NotAfter: Aug 28 18:05:52 2027 GMT
-----BEGIN CERTIFICATE-----
MIIFPzCCAyegAwIBAgIUDWReHWch8xYc1FUdyB3B/CRmFLAwDQYJKoZIhvcNAQEL
BQAwghcyCzAJBgNVBAYTA1VTMRUwEwYDVR0QDAxOZn5uc3lscmEuaWExEzARBQNV
-----END CERTIFICATE-----
```



Your connection is not private

Attackers might be trying to steal your information from **localhost** (for example, passwords, messages, or credit cards). [Learn more about this warning](#)

NET::ERR_CERT_AUTHORITY_INVALID

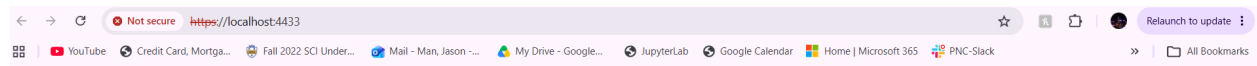


Turn on [enhanced protection](#) to get Chrome's highest level of security

Advanced

Back to safety

Step 7: Trust your CA and remove the warning

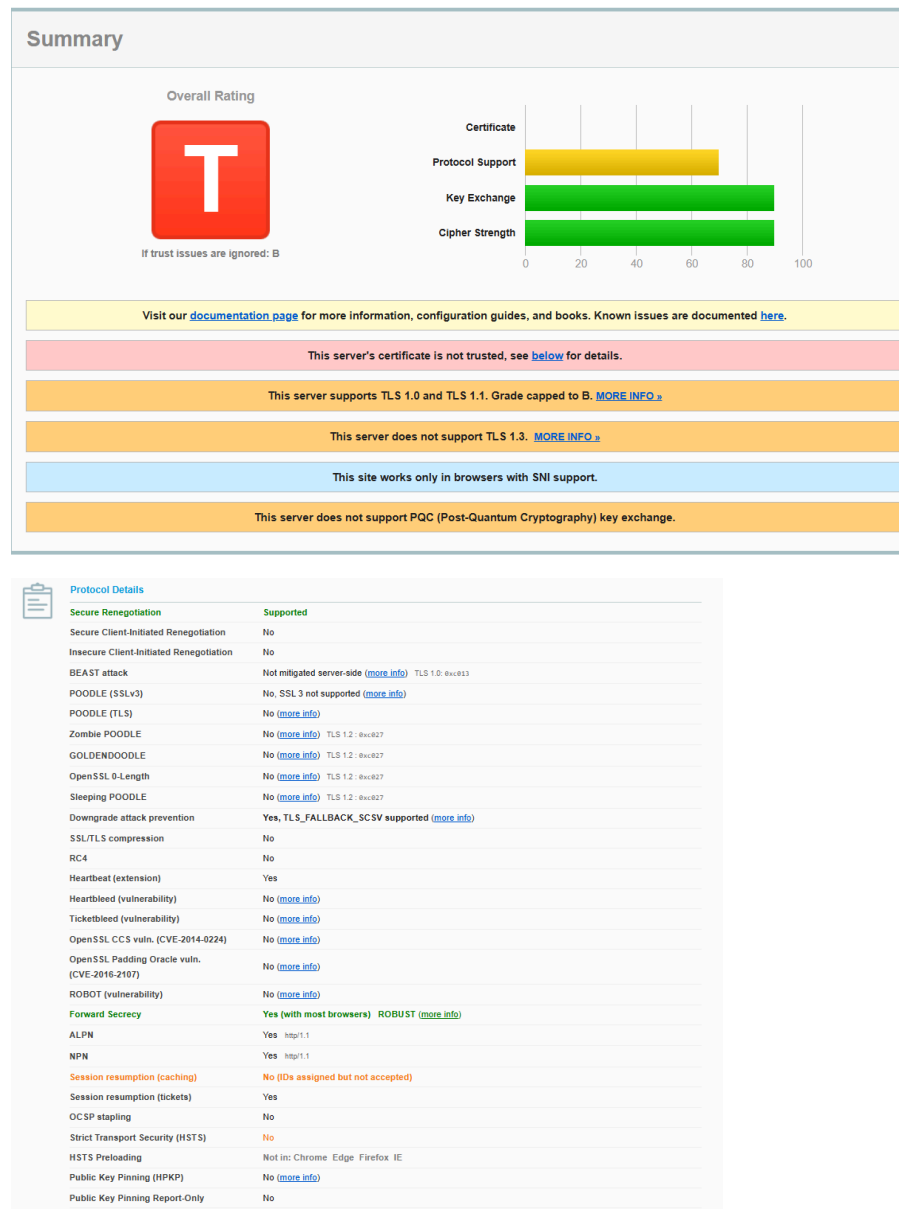


Step 8: Final check and validation

```
manom@HomePC2026 MINGW64 ~
$ openssl s_client -connect localhost:4433 -showcerts
Connecting to 127.0.0.1
CONNECTED(00000208)
Can't use SSL_get_servername
depth=0 C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=SOC, CN=localhost, emailAd
dress=user2@example.com
verify error:num=20:unable to get local issuer certificate
verify return:1
depth=0 C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=SOC, CN=localhost, emailAd
dress=user2@example.com
verify error:num=21:unable to verify the first certificate
verify return:1
depth=0 C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=SOC, CN=localhost, emailAd
dress=user2@example.com
verify return:1
---
Certificate chain
 0 s:C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=SOC, CN=localhost, emailAdre
ss=user2@example.com
  i:C=US, ST=Pennsylvania, L=Pittsburgh, O=Meridian Trust Banking, OU=IT, CN=MeridianTrust-Lab-CA,
emailAddress=user@example.com
  a:PKEY: RSA, 2048 (bit); sigalg: sha256withRSAEncryption
  v:NotBefore: Aug 28 18:02:58 2026 GMT; NotAfter: Aug 28 18:02:58 2027 GMT
-----BEGIN CERTIFICATE-----
MIIFfZCCAv+gAwIBAgIUEmpTTK+QzF1ySHlg45uBNBDFdUQwDQYJKoZIhvcNAQEL
BQAwgacxCzAJBgNVBAYTA1VTMRUwEwYDVQQIDAxQZW5uc3lsdmFuawExEzARBgNV
BACMC1BpdHRzYnVyZ2gxHZAAdBgNVBAoMFk1lcm1kawFuIFRydXN0IEJhbmtpbmcx
CzAJBgNVBAsMAk1UMR0wGwYDVQQDDBRNZXJpZG1hb1RydXN0LUXhYi1DQTEfMB0G
CSqGS1b3DQEJARYQdXNlckBleGFtcGx1LmNvbTAeFw0yNjA4MjgxODAyNThaFw0y
NzA4MjgxODAyNThaMIGeMQswCQYDVQQGEwJVUzEVMBMGA1UECAwMUGVubnN5bHZh
bm1hMRMwEQYDVQQHDApQaXR0c2J1cmdoMR8wHQYDVQQKDBZBNZjZG1hb1BUCnVz
```

Part 2, Day 5: TLS Misconfiguration Investigation

Step 1: Run the free online scanner



Certificate #1: RSA 2048 bits (SHA256withRSA)



Server Key and Certificate #1

Subject	*.badssl.com
	Fingerprint SHA256: ba105ce02bac76888ecee47cd4eb7941653e9ac993b61b2eb3dcc82014d21b4f
	Pin SHA256: 9SLkksvzMYj8t+52lp5ze/hY0CFHyLSPQzSpYYlBm8=
Common names	*.badssl.com
Alternative names	*.badssl.com badssl.com
Serial Number	4ae79549fa9abe3f100f17a478e16909
Valid from	Thu, 09 Apr 2015 00:00:00 UTC
Valid until	Sun, 12 Apr 2015 23:59:59 UTC (expired 11 years and 4 months ago) EXPIRED
Key	RSA 2048 bits (e 65537)
Weak key (Debian)	No
Issuer	COMODO RSA Domain Validation Secure Server CA AIA: http://crl.comodoca.com/COMODORSADomainValidationSecureServerCA.crt
Signature algorithm	SHA256withRSA
Extended Validation	No
Certificate Transparency	No
OCSP Must Staple	No
Revocation information	CRL: http://crl.comodoca.com/COMODORSADomainValidationSecureServerCA.crl OCSP: http://ocsp.comodoca.com
Revocation status	Unchecked (only trusted certificates can be checked)
DNS CAA	No (more info)
Trusted	No NOT TRUSTED (Why?) Mozilla Apple Android Java Windows



Cipher Suites

TLS 1.2 (suites in server-preferred order)

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f) ECDH secp256r1 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030) ECDH secp256r1 (eq. 3072 bits RSA) FS

TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02e) DH 2048 bits FS

TLS_DHE_RSA_WITH_AES_256_GCM_SHA384 (0xc031) DH 2048 bits FS

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (0xc027) ECDH secp256r1 (eq. 3072 bits RSA) FS WEAK

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013) ECDH secp256r1 (eq. 3072 bits RSA) FS WEAK

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (0xc028) ECDH secp256r1 (eq. 3072 bits RSA) FS WEAK

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014) ECDH secp256r1 (eq. 3072 bits RSA) FS WEAK

TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 (0xc02f) DH 2048 bits FS WEAK

TLS_DHE_RSA_WITH_AES_128_CBC_SHA (0xc013) DH 2048 bits FS WEAK

TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (0xc030) DH 2048 bits FS WEAK

TLS_DHE_RSA_WITH_AES_256_CBC_SHA (0xc014) DH 2048 bits FS WEAK

TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA (0xc012) ECDH secp256r1 (eq. 3072 bits RSA) FS WEAK

TLS_RSA_WITH_AES_128_GCM_SHA256 (0xc02f) WEAK

TLS_RSA_WITH_AES_256_GCM_SHA384 (0xc030) WEAK

TLS_RSA_WITH_AES_128_CBC_SHA256 (0xc027) WEAK

TLS_RSA_WITH_AES_256_CBC_SHA256 (0xc028) WEAK

TLS_RSA_WITH_AES_128_CBC_SHA (0xc013) WEAK

TLS_RSA_WITH_AES_256_CBC_SHA (0xc014) WEAK

TLS_RSA_WITH_3DES_EDE_CBC_SHA (0xc012) WEAK

TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA (0xc098) DH 2048 bits FS WEAK

TLS_RSA_WITH_CAMELLIA_256_CBC_SHA (0xc084) WEAK

TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA (0xc045) DH 2048 bits FS WEAK

TLS_RSA_WITH_CAMELLIA_128_CBC_SHA (0xc041) WEAK

TLS 1.1 (suites in server-preferred order)

TLS 1.0 (suites in server-preferred order)

Step 2: Confirm the same findings from the command line

```
kar@redborg:~/work4-CTF-1-10$ openssl s_client -connect expired.badssl.com:443
Connecting to 104.154.89.105
CONNECTED(000001C4)
depth=2 C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RSA Certification Authority
verify return:1
depth=1 C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RSA Domain Validation Secure Server CA
verify return:1
depth=0 OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
verify error:num=10:certificate has expired
notAfter=Apr 12 23:59:59 2015 GMT
verify return:1
depth=0 OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
notAfter=Apr 12 23:59:59 2015 GMT
verify return:1
---
Certificate chain
 0 s:OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
   i:C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RSA Domain Validation Secure Server CA
   a:PKEY: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
   v:NotBefore: Apr  9 00:00:00 2015 GMT; NotAfter: Apr 12 23:59:59 2015 GMT
 1 s:C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RSA Domain Validation Secure Server CA
   i:C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RSA Certification Authority
   a:PKEY: RSA, 2048 (bit); sigalg: sha384WithRSAEncryption
   v:NotBefore: Feb 12 00:00:00 2014 GMT; NotAfter: Feb 11 23:59:59 2029 GMT
 2 s:C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RSA Certification Authority
   i:C=SE, O=AddTrust AB, OU=AddTrust External TTP Network, CN=AddTrust External CA Root
   a:PKEY: RSA, 4096 (bit); sigalg: sha384WithRSAEncryption
   v:NotBefore: May 30 10:48:38 2000 GMT; NotAfter: May 30 10:48:38 2020 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIFSzCCBD0GAWIBAgIQSueVSFqavj8QDxeke0FpCTANBgkqhkiG9w0BAQsFADCB
kDELMAkGA1UEBhMCRC0xGzAZBgNVBAgTEkdyZWFOZXIgTWFuY2hlc3RlcjEQAQA4G
A1UEBxMHU2FsZm9yZDEaMBgGA1UEChMRQ09NTORPIENBIExpbWl0ZWQxNjA0BgNV
BAMTLUNPTU9ETyBSU0EgRG9tYWluIFZhbG1kYXRpb24uU2VjdXJlIFNlcnZlcjBD
QTAEFw0xNTA0MDkwMDAwMDBaFw0xNTA0MTIyMzU5NTIaMFkxITAFBgNVBAsTGERv
bWVpb250cm9sIFZhbG1kYXR1ZDEdMBsGA1UECzMUUG9zaXRpdmVU0wgV21s
ZGNhcmQxFTATBgNVBAMUDCouYmFkc3NsLmNvbTCCASIwDQYJKoZIhvcNAQEBBQAD
ggEPADCCAQoCggEBAMIE7PiM7qTCs9hQ1XBYzJMY61vqaEmwIrX51Z6xKyx2PmzA
-----END CERTIFICATE-----
```

Step 3: Assign a severity

High severity

An attacker could use a rogue certificate to reroute web traffic

To fix this vulnerability the expired certificate would have to be replaced and ensure the private key is secure.

Step 4: Key management review

The first thing wrong is that it is stored in the same configuration file. The second is that it is unencrypted and the third problem is that tokenization isn't enabled.

Generally it is better to store keys in an HSM where it will be encrypted, tokenized, and the process of the key management lifecycle is more automated.

Tokenization would definitely reduce this application's compliance exposure. This is because there are card numbers being stored which are considered private important meaningful data. By enabling tokenization the important card information will not be readable to anyone.

Step 5: Post- quantum awareness

“Harvest now, decrypt later” is where threat actors intercept or keep encrypted data that they currently can’t exploit with current processing technology. They then wait until there are advancements in quantum computing so they can use it to break current encryption standards.

One type of long-lived bank data that would be at risk is card numbers and any other sensitive data.